

UNIVERSIDAD POLITÉCNICA SALESIANA

CARRERA DE INGENIERÍA AUTOMOTRIZ

SEDE QUITO – CAMPUS SUR

INFORME DE PRÁCTICA

AUTOTRONICA

PERÍODO 68

	UNIVERSIDAD POLITÉCNICA SALESIANA CARRERA DE INGENIERÍA AUTOMOTRIZ SEDE QUITO – CAMPUS SUR	Ingeniería Automotriz SEDE QUITO
INFORME DE PRÁCTICA		
Versión: 1.0	Código: CIAUT-DOC-INF RESOLUCIÓN No. 0335-009-2026-04-10.	Página 2 de 12

Asignatura: Autotrónica			
Nombre de la práctica: Recuperación segundo interciclo			
Integrante (s): Maryori Baño Andersson Lema Edison Casa			Fecha de realización de la práctica: 23/07/2026
Carrera: Ingeniería Automotriz	Período: P68 Nivel: 6to Grupo: G1	Grupo de trabajo: N° 3	Fecha de presentación del informe: 30/07/2026
Objetivos			
General: Desarrollar una aplicación móvil que permita controlar y monitorear el sistema de iluminación de la tarjeta de prácticas mediante comunicación Bluetooth entre el teléfono y el microcontrolador Arduino, utilizando la tarjeta MCP2515 para la transmisión y recepción de mensajes sobre la red CAN BUS. Específicos: • Diseñar una aplicación móvil que permita controlar y monitorear el sistema de iluminación de la tarjeta de prácticas mediante una interfaz intuitiva. • Implementar la comunicación Bluetooth entre el dispositivo móvil y el microcontrolador Arduino, integrando la tarjeta MCP2515 para la transmisión y recepción de mensajes a través de la red CAN BUS. • Evaluar el funcionamiento del sistema mediante pruebas de control y monitoreo del sistema de iluminación, verificando la correcta comunicación entre la aplicación móvil, Arduino y la red CAN BUS.			
Materiales, herramientas y equipos			
Herramientas y equipos:	Materiales e insumos:	Material Didáctico:	Equipo de Seguridad:

	UNIVERSIDAD POLITÉCNICA SALESIANA CARRERA DE INGENIERÍA AUTOMOTRIZ SEDE QUITO – CAMPUS SUR	Ingeniería Automotriz SEDE QUITO
INFORME DE PRÁCTICA		
Versión: 1.0	Código: CIAUT-DOC-INF RESOLUCIÓN No. 0335-009-2026-04-10.	Página 3 de 12

• Multímetro.	• Módulo Bluetooth (HC-05, HC-06 o equivalente). • Arduino UNO. • Módulo CAN BUS MCP2515.	• Banco didáctico UniTrain de Luces-Nulle	• Mandil.	
---	---	---	---	--

Marco Teórico

1. El Sistema CAN BUS (Controller Area Network)

El sistema CAN BUS es un estándar de comunicación diseñado originalmente por Robert Bosch GmbH para su uso en automóviles. Su función principal es permitir que las diferentes unidades de control (como la del motor) y sus componentes electrónicos se comuniquen entre sí de manera eficiente sin necesidad de un ordenador central (Calle Verdugo & Loja Chimborazo, 2024). Este protocolo utiliza un bus de dos cables para la recepción y envío de datos digitales, lo que ayuda a evitar errores o la saturación del sistema al gestionar el flujo de información.

2. El Controlador CAN MCP2515 y el Transceptor TJA1050

Para que un microcontrolador (como Arduino) pueda "hablar" el lenguaje de un vehículo, se utiliza el módulo MCP2515. Este es un controlador CAN independiente que simplifica la conexión con el bus de datos del automóvil (Rajguru Electronics, s. f.). El módulo consta de tres bloques principales: el módulo CAN (que gestiona los mensajes y filtros), la lógica de control y el bloque de interfaz SPI para comunicarse con el microcontrolador. Este controlador trabaja en conjunto con el chip TJA1050, que actúa como el enlace físico entre el controlador y los cables del bus CAN del vehículo, permitiendo velocidades de hasta 1 Mb/s.

3. Tecnología Bluetooth y Módulo HC-05

La comunicación Bluetooth es una tecnología inalámbrica de corto alcance que opera en la banda de 2,4 GHz (Electronics Hub, s. f.). El módulo HC-05 es ampliamente utilizado en proyectos, porque permite reemplazar las conexiones seriales por cable con un enlace inalámbrico estable. Este dispositivo puede funcionar en dos modos:

- **Modo de Datos:** Para enviar y recibir información de forma normal entre dispositivos (como un celular y un Arduino).
- **Modo de Comandos AT:** Utilizado para configurar parámetros como el nombre del dispositivo, la contraseña y la velocidad de comunicación. Tiene un alcance típico de hasta 10 metros, lo que lo hace ideal para el monitoreo local de sistemas vehiculares.

4. Integración con Arduino y Aplicaciones Móviles

	UNIVERSIDAD POLITÉCNICA SALESIANA CARRERA DE INGENIERÍA AUTOMOTRIZ SEDE QUITO – CAMPUS SUR	Ingeniería Automotriz SEDE QUITO
INFORME DE PRÁCTICA		
Versión: 1.0	Código: CIAUT-DOC-INF RESOLUCIÓN No. 0335-009-2026-04-10.	Página 4 de 12

El microcontrolador Arduino Uno actúa como el cerebro del sistema, adquiriendo los datos del bus CAN a través del MCP2515 mediante el protocolo de comunicación SPI (Allelco, s. f.). Para visualizar la información en tiempo real, se suelen utilizar aplicaciones móviles desarrolladas en plataformas como RoboBoy, la cual permite crear interfaces gráficas mediante un sistema de bloques fácil de programar. Esta integración permite monitorear parámetros críticos del motor a través del puerto OBD-II del vehículo.

Identificación de la Necesidad y del Problema

La necesidad de la práctica:

Existe la necesidad de comprender el funcionamiento de la comunicación entre una aplicación móvil, el microcontrolador Arduino y la red CAN BUS mediante el módulo MCP2515, con el fin de fortalecer los conocimientos sobre el control y monitoreo de sistemas electrónicos automotrices utilizando tecnologías de comunicación inalámbrica como Bluetooth.

El problema a resolver:

Se desconoce cómo implementar un sistema que permita controlar y monitorear el sistema de iluminación de una tarjeta de prácticas desde una aplicación móvil mediante comunicación Bluetooth, garantizando la correcta transmisión y recepción de mensajes a través de la red CAN BUS utilizando el módulo MCP2515 y el microcontrolador Arduino.

INFORME DE PRÁCTICA

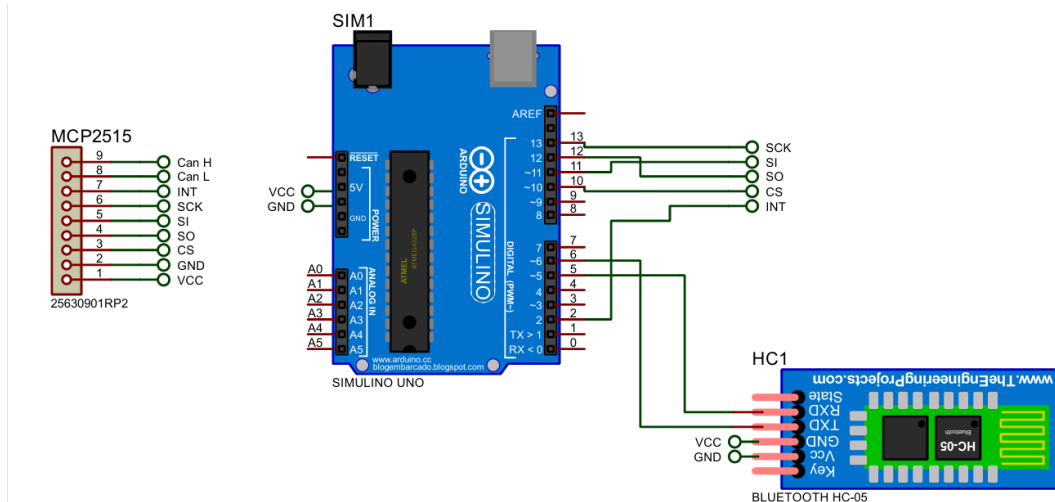
Versión: 1.0

Código: CIAUT-DOC-INF
RESOLUCIÓN No. 0335-009-2026-04-10.

Página 5 de 12

Procedimiento

Diagrama de conexión.



Descripción del funcionamiento.

Tabla de IDs utilizados y DLC de cada mensaje.

Comando	ID CAN (Hex)	Longitud (DLC)	Datos (Bytes Hex)
PARQ_ON	0x782	3	0x1F, 0x40, 0x48
PARQ_OFF	0x782	3	0x00, 0x40, 0x48
FREN_ON	0x783	1	0x08
FREN_OFF	0x783	1	0x00
BAJAS_ON	0x784	1	0x61
BAJAS_OFF	0x784	1	0x00
ALTAS_ON	0x784	1	0x63
ALTAS_OFF	0x784	1	0x61

INFORME DE PRÁCTICA

Versión: 1.0

Código: CIAUT-DOC-INF
RESOLUCIÓN No. 0335-009-2026-04-10.

Página 6 de 12

DIR_DER_ON	0x782	3	0x3A, 0x40, 0x80
DIR_DER_OFF	0x782	3	0x00, 0x40, 0x80
DIR_IZQ_ON	0x782	3	0x25, 0x40, 0x80
DIR_IZQ_OFF	0x782	3	0x00, 0x40, 0x80
PTA_DER_ON	0x78D	3	0x01, 0x00, 0x00
PTA_DER_OFF	0x78D	3	0x00, 0x00, 0x00
PTA_IZQ_ON	0x78C	3	0x01, 0x00, 0x00
PTA_IZQ_OFF	0x78C	3	0x00, 0x00, 0x00
ABIERTO	0x78A	1	0x0A
CERRADO	0x78A	1	0x05

Evidencias fotográficas o capturas de pantalla.

Obtención de los IDs



Experimentos

Antes de realizar el primer experimento, por favor, lee con atención las siguientes instrucciones. **Aquí** también encontrarás indicaciones para el manejo de los instrumentos virtuales.

Conexión del sistema de experimentación al PC

Estas notas son válidas para **todos** los experimentos.

Para que se pueda establecer la comunicación entre el sistema de experimentación y el PC, se deben cumplir las siguientes condiciones:

- Se requiere la instalación del controlador del puerto **CAN MONITOR**.
- Se debe conectar la interfaz UniTrain SO4203-2A.
- La interfaz UniTrain SO4203-2A se debe conectar a:
- Las dos tarjetas CAN deben encontrarse insertadas en:

El software no debe funcionar en modo de simulación (esta opción se selecciona al inicio del programa).

Ajustes básicos

Estos ajustes son válidos para todos los experimentos **con excepción** de aquellos dedicados a la localización de fallos, instancia en la que rigen **otras condiciones**.

- Todos los experimentos deben ejecutarse en el modo de baja velocidad ("Low Speed"). Este modo se selecciona en el monitor CAN, en el recuadro rotulado con la palabra "MODE". Si se presentaran excepciones, se hará especial énfasis en el procedimiento a seguir.
- El osciloscopio debe funcionar siempre en el modo "X1".
- Cuando se requiera activar el osciloscopio, se deben emplear siempre los siguientes ajustes:
 - Disparo en el canal A
 - Disparo en flanco ascendente

INFORME DE PRÁCTICA

Versión: 1.0

Código: CIAUT-DOC-INF
RESOLUCIÓN No. 0335-009-2026-04-10.

Página **7** de **12**

Código

```
// Parqueo
{"parkingOn", 0x782, 3, {0x1F,0x40,0x48}},
{"parkingOff",0x782, 3, {0x00,0x40,0x48}},

// Freno
{"brakeOn", 0x783, 1, {0x08}},
{"brakeOff",0x783, 1, {0x00}},

// Luces bajas
{"lowBeamOn", 0x784, 1, {0x61}},
{"lowBeamOff",0x784, 1, {0x00}},

// Luces altas
{"highBeamOn", 0x784, 1, {0x63}},
{"highBeamOff",0x784, 1, {0x61}},

// Direccional derecha
{"rightSignalOn",0x782,3,{0x3A,0x40,0x80}},
{"rightSignalOff",0x782,3,{0x00,0x40,0x80}},

// Direccional izquierda
{"leftSignalOn",0x782,3,{0x25,0x40,0x80}},
{"leftSignalOff",0x782,3,{0x00,0x40,0x80}},

// Puerta derecha
{"rightDoorOpen",0x78D,3,{0x01,0x00,0x00}},
{"rightDoorClose",0x78D,3,{0x00,0x00,0x00}},

// Puerta izquierda
{"leftDoorOpen",0x78C,3,{0x01,0x00,0x00}},
{"leftDoorClose",0x78C,3,{0x00,0x00,0x00}},
```



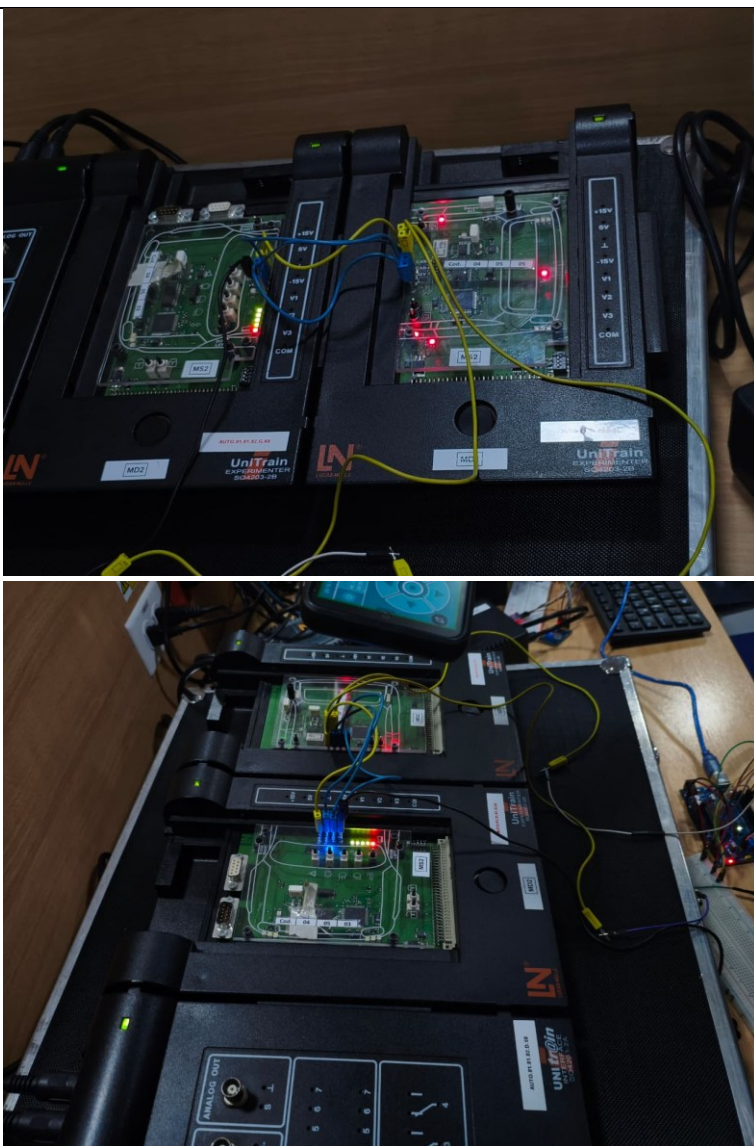
Imagen de funcionamiento

INFORME DE PRÁCTICA

Versión: 1.0

Código: CIAUT-DOC-INF
RESOLUCIÓN No. 0335-009-2026-04-10.

Página 8 de 12

**Funcionamiento****Descripción del Funcionamiento del Sistema**

El sistema opera como un puente traductor entre el protocolo de comunicación inalámbrica Bluetooth y el bus de datos industrial CAN. Su flujo de funcionamiento y la arquitectura del programa en Arduino se estructuran de la siguiente manera:

	UNIVERSIDAD POLITÉCNICA SALESIANA CARRERA DE INGENIERÍA AUTOMOTRIZ SEDE QUITO – CAMPUS SUR	Ingeniería Automotriz SEDE QUITO
INFORME DE PRÁCTICA		
Versión: 1.0	Código: CIAUT-DOC-INF RESOLUCIÓN No. 0335-009-2026-04-10.	Página 9 de 12

Librerías e Inicialización de Hardware

- **mcp_can.h y SPI.h:** Gestionan la comunicación por bus SPI entre el Arduino y el controlador CAN MCP2515.
- **SoftwareSerial.h:** Define un puerto serie secundario en los pines digitales 3 (RX) y 4 (TX) para la recepción serie desde el módulo Bluetooth.
- **CAN_CS_PIN 10:** Asigna el pin 10 como la línea *Chip Select* (CS) para habilitar el chip MCP2515.

Estructuras de Datos y Tablas de Comandos

El código organiza los mensajes mediante tres estructuras estructuradas:

- **CanCommand commands[]:** Contiene la base de datos de los comandos individuales (luces, parqueo, direccionales). Cada entrada incluye la etiqueta interna de la variable, el ID del bus CAN en formato hexadecimal (ej. 0x782), la longitud DLC (1 o 3 bytes) y el arreglo de bytes en hexadecimal.
- **CanFrame lockFrames[] y unlockFrames[]:** Almacenan ráfagas de tramas múltiples necesarias para ejecutar operaciones complejas como el bloqueo o desbloqueo total del vehículo, inyectando secuencias consecutivas a los IDs 0x78A, 0x78C, 0x78D, 0x78E, 0x78F y 0x790.
- **CommandMap commandMap[]:** Vincula las cadenas numéricas provenientes de la app RoboBoy vía Bluetooth con las etiquetas de comando del programa (por ejemplo, la entrada "1" activa "parkingOn", mientras que "10" ejecuta "parkingOff").

Configuración Inicial (setup)

- Inicia la comunicación serie a 9600 baudios tanto para la terminal de monitoreo como para la recepción Bluetooth.
- Inicializa el controlador MCP2515 configurando la red CAN a una velocidad de 125 Kbps con un cristal de 8 MHz (CAN_125KBPS, MCP_8MHZ), coincidiendo con el estándar de baja velocidad (*Low Speed*) empleado en las tarjetas de entrenamiento del sistema UniTrain.
- Establece el modo operativo en normal (MCP_NORMAL) para permitir el tráfico continuo de mensajes.

Bucle Principal (loop)

- Realiza un monitoreo permanente del puerto serie Bluetooth mediante BT.available().

	UNIVERSIDAD POLITÉCNICA SALESIANA CARRERA DE INGENIERÍA AUTOMOTRIZ SEDE QUITO – CAMPUS SUR	Ingeniería Automotriz SEDE QUITO
INFORME DE PRÁCTICA		
Versión: 1.0	Código: CIAUT-DOC-INF RESOLUCIÓN No. 0335-009-2026-04-10.	Página 10 de 12

- Al detectar una entrada, lee la cadena hasta el caracter de salto de línea (\n), elimina espacios en blanco mediante .trim() y pasa la instrucción a la función de procesamiento executeCommand().

Lógica de Ejecución y Secuencias Visuales

Al procesar una orden, el programa evalúa los siguientes escenarios:

- **Comandos Especiales ("99" y "0"):**
 - Si recibe el código "99", envía la secuencia completa lockFrames para bloquear el sistema y ejecuta la rutina lockSequence(), la cual mantiene encendidas las direccionales durante 3 segundos como retroalimentación visual.
 - Si recibe el código "0", envía la secuencia unlockFrames para liberar seguros y ejecuta unlockSequence(), haciendo destellar las direccionales tres veces seguidas.
- **Comandos Individuales:** Si recibe cualquier otro código (ej. "3", "5", "30"), busca la coincidencia dentro del mapa commandMap y llama a la función sendCommand().

Transmisión de Tramas al Bus CAN (sendMsgBuf)

El envío de datos hacia el bus físico en las funciones sendCommand() y sendFrameGroup() utiliza el método nativo:

CAN.sendMsgBuf(id,0,dlc,data)

Este comando inyecta directamente el paquete de datos en el bus del UniTrain. Se aplica un retardo de 10 ms (delay(10)) entre transmisiones sucesivas para evitar colisiones y asegurar la estabilidad en la recepción de la tarjeta de iluminación.

Conclusiones

- Se desarrolló correctamente una aplicación móvil capaz de controlar y monitorear el sistema de iluminación del banco didáctico mediante comunicación Bluetooth con Arduino, cumpliendo el objetivo de implementar una interfaz funcional para el usuario.
- Se logró integrar el módulo Bluetooth HC-05 con el Arduino Uno y la tarjeta MCP2515, permitiendo la transmisión y recepción de mensajes a través de la red CAN BUS. Esto permitió comprender el funcionamiento de la comunicación entre dispositivos y el intercambio de tramas CAN.

	UNIVERSIDAD POLITÉCNICA SALESIANA CARRERA DE INGENIERÍA AUTOMOTRIZ SEDE QUITO – CAMPUS SUR	Ingeniería Automotriz SEDE QUITO
INFORME DE PRÁCTICA		
Versión: 1.0	Código: CIAUT-DOC-INF RESOLUCIÓN No. 0335-009-2026-04-10.	Página 11 de 12

- Las pruebas realizadas demostraron que el sistema respondió correctamente a los comandos enviados desde la aplicación móvil, verificando el correcto control de las luces y la estabilidad de la comunicación Bluetooth y CAN BUS. Con ello se cumplieron los objetivos planteados para la práctica y se fortalecieron los conocimientos sobre programación, comunicación serial y redes de comunicación automotriz.

Recomendaciones

- Verificar previamente la configuración de los módulos Bluetooth y MCP2515, así como las conexiones del cableado, para evitar errores de comunicación durante el desarrollo de la práctica.
- Realizar pruebas individuales de la comunicación Bluetooth y de la red CAN BUS antes de integrar todo el sistema, con el fin de identificar y corregir fallas de manera más rápida.
- Mantener una identificación clara de los identificadores (ID) y mensajes CAN utilizados en la programación para facilitar el seguimiento de la transmisión y recepción de datos.
- Documentar los resultados obtenidos en cada prueba, incluyendo capturas de la aplicación móvil, del monitor serie de Arduino y del funcionamiento del sistema de iluminación, para facilitar el análisis y la verificación del correcto funcionamiento.
- Utilizar una fuente de alimentación estable y revisar periódicamente el estado de los componentes y conexiones de la tarjeta de prácticas, evitando interrupciones o fallos durante la ejecución del sistema.

Bibliografía

- Allelco. (s.f.). *HC-05 Bluetooth Module Guide: Features, AT Commands, and Arduino Projects*. <https://www.allelcoelec.com/blog/HC-05-Bluetooth-Module-Guide-Features,AT-Commands,and-Arduino-Projects.html>
- Calle Verdugo, C. A., & Loja Chimborazo, S. J. (2024). *Implementación de un sistema de monitoreo con aplicación y visualización digital de los parámetros de funcionamiento del motor Kia Rio Xcite 2008 1.4 CC* [Trabajo de titulación de pregrado, Universidad Politécnica Salesiana]. Repositorio Institucional de la

	UNIVERSIDAD POLITÉCNICA SALESIANA CARRERA DE INGENIERÍA AUTOMOTRIZ SEDE QUITO – CAMPUS SUR	Ingeniería Automotriz SEDE QUITO
INFORME DE PRÁCTICA		
Versión: 1.0	Código: CIAUT-DOC-INF RESOLUCIÓN No. 0335-009-2026-04-10.	Página 12 de 12

Universidad Politécnica Salesiana. https://dspace.ups.edu.ec/bitstream/123456789/25680/1/UPS-CT010747.pdf, • Electronics Hub. (s.f.). <i>Arduino MCP2515 CAN Bus Interface Tutorial.</i> https://how2electronics.com/interfacing-mcp2515-can-bus-module-with-arduino/, • Rajguru Electronics (I) Pvt. Ltd. (s.f.). <i>MCP2515 CAN Bus Module.</i> https://www.mactronica.com.co/modulo-can-bus-mcp2515,
--